

Miguel Core Unified Daemon

Eigenständiges technisches Deep-Dive-Dossier

Miguel Core Unified Daemon

Ich wollte meine vielen einzelnen Agenten-, Voice-, Memory- und Tool-Skripte nicht als lose Sammlung behalten. Miguel Core war mein Versuch, daraus eine beobachtbare lokale Kontrollschicht zu machen.

- Zeigt, dass ich viele einzelne KI-Experimente nicht nur gesammelt, sondern zu einer lokalen Kontrollschicht mit Routen, State, Health, Worker-Delegation und Grenzen verdichtet habe.
- Lokale experimentelle Kontrollschicht, nicht als fertiges kommerzielles Produkt formuliert.

Was ich mir dabei gedacht habe

Miguel Core entstand aus einer sehr praktischen Frustration: Ich hatte zu viele starke Teile als einzelne Skripte und Services. Ich wollte daraus eine lokale Kontrollschicht machen, die beobachtbar, routbar und verständlicher ist.

Was ich gebaut habe

- Ich denke Agenten nicht als einzelne Prompts, sondern als Betriebssystem-nahe Services.
- Ich kann verstreute Prototypen in eine zentrale Kontrollfläche zusammenführen.
- Ich habe früh über Health, State, Tool-Oberflächen und Autonomiegrenzen nachgedacht.

Mechanik im Detail

- Miguel Core lässt sich am stärksten als lokale Agenten-Betriebsschicht erklären. Chat, Streaming, Voice, State, Memory, Prediction, Modellrouting, Orchestrierung und Tool-Endpunkte werden in eine beobachtbare Kontrollschicht gezogen.
- Die Route Surface ist wichtig, weil sie zeigt, dass die Arbeit von Einzelskripten in Richtung System mit Health, State, Conversation Persistence und Worker Delegation gegangen ist.
- Startup- und Handover-Dokumente gehören zur Evidenz: Sie definieren Boot Checks, Betriebsmodi, Quality Anchors, Autonomy Rings und State Probes.

Architektur

- Ein Python-Daemon bündelt Chat, Streaming, Voice, State, Vault, Knowledge und Tools.
- Routen verbinden Memory Broker, CASS, Vault Watcher, Prediction, Metacognition und macOS Bridge.
- Orchestra delegiert Aufgaben an Worker und verfolgt Status, Timeout, Ergebnis und Abbruch.
- Start- und Handover-Dateien dokumentieren Modellrouting, Boot Checks und Autonomie-Ringe.

Evidenzcluster

- Miguel-Core-Daemon als Konsolidierung früherer Daemon- und Command-Flächen in eine lokale Kontrollschicht.
- Route Map für Chat, Streaming, Voice, State, Vault, Knowledge, Prediction, Cascade, Agents, Orchestra, macOS/Tool-Control, Conversations und Health.
- Orchestra-Worker-Modul mit asynchroner Delegation, Timeouts, Cancellation, Status und Result Capture.

- Startup- und Handover-Dokumente mit Boot-Probes, Routing Policies, Quality Gates, Betriebsmodi und Autonomy Rings.

Claim-zu-Evidenz-Karte

Die öffentliche Version soll stark wirken, weil sie begrenzt ist, nicht weil sie übertreibt.

Claim	Evidenz	Grenze
Zeigt, dass ich viele einzelne KI-Experimente nicht nur gesammelt, sondern zu einer lokalen Kontrollschicht mit Routen, State, Health, Worker-Delegation und Grenzen verdichtet habe.	Miguel-Core-Daemon als Konsolidierung früherer Daemon- und Command-Flächen in eine lokale Kontrollschicht.; Route Map für Chat, Streaming, Voice, State, Vault, Knowledge, Prediction, Cascade, Agents, Orchestra, macOS/Tool-Control, Conversations und Health.	Lokale experimentelle Kontrollschicht, nicht als fertiges kommerzielles Produkt formuliert.
Die Arbeit ist wertvoll, weil sie Mechanik zeigt, nicht nur Oberfläche.	Ein Python-Daemon bundelt Chat, Streaming, Voice, State, Vault, Knowledge und Tools.; Routen verbinden Memory Broker, CASS, Vault Watcher, Prediction, Metacognition und macOS Bridge.	Frische öffentliche Demos oder Benchmarks wären die nächste Beweisschicht.

Senior-Level-Signal

Zeigt, dass ich viele einzelne KI-Experimente nicht nur gesammelt, sondern zu einer lokalen Kontrollschicht mit Routen, State, Health, Worker-Delegation und Grenzen verdichtet habe.

- Architekturkonsolidierung: Fragmentierung erkennen und viele Flächen in eine kontrollierbare Schicht bringen.
- Operative Reife: Boot Probes und Handover-Dokumente zeigen, dass das System betrieben werden sollte, nicht nur als Idee existiert.
- Grenzreife: Autonomy Rings trennen, was das System darf, von dem, was Nutzerfreigabe braucht.

Skeptische Reviewer-Fragen

- Was ist hier wirklich gebaut und was ist noch Design? Antwort: Status und Grenze stehen im Dossier direkt beim Fall Miguel Core Unified Daemon.
- Welche private Evidenz wurde bewusst nicht veröffentlicht? Antwort: lokale Pfade, Credentials, private Kanaldetails und vertrauliche Rohtexte bleiben draußen.
- Warum ist das relevant? Antwort: Der Fall zeigt einen wiederverwendbaren Baustein für Agenten, Tool-Nutzung, Memory, Routing, Validierung oder High-Stakes-Governance.
- Was wäre der nächste belastbare Beweis? Antwort: synthetische Demo, Audit Trace, Benchmark oder externe Review, je nach Fall.

Lernpunkte und Grenzen

Lokale experimentelle Kontrollschicht, nicht als fertiges kommerzielles Produkt formuliert.

- Eine Route zu haben ist nicht dasselbe wie eine Route produktionsreif zu haerten.
- Lokale Systeme brauchen trotzdem Beobachtbarkeit, klare Rechte und Fehlergrenzen.
- Autonomie-Ringe sind eine hilfreiche Sprache für sichere und unsichere Aktionen.

Nächste Härtung / nächster Projektschritt

- Stabile Public-API aus dem breiten internen Daemon herauslösen.
- Endpoint-Rechte und Risiko-Labels maschinenlesbar dokumentieren.
- Health Reports für alle Hauptmodule erzeugen.
- Eine öffentliche Demo-Route-Map mit privaten Aktionen als sichere Stubs zeigen.
- Endpoint-Metadaten ergänzen: Permission, betroffene Daten, Modell, Timeout und Audit Trace.
- Einen synthetischen Health Report veröffentlichen, der pro Route den Bereitschaftszustand ausweist.